

PYTHON

Complete Handbook

From Zero to Job-Ready Developer

Beginner → Intermediate → Advanced

15 Chapters · 3 Real Projects · 175 Interview Q&A · Cheat Sheets

2025 Edition

Zero Experience
Required

Practical Code
Examples

Interview Q&A
Included

PREFACE

Welcome to the Python Complete Handbook

This handbook was written for **you** — someone who has never written a single line of code but wants to become a confident Python developer.

Python is the world's most popular programming language. It powers Google's search engine, Netflix's recommendation system, NASA's data analysis pipelines, and Instagram's entire backend. Learning Python opens doors to software development, data science, artificial intelligence, automation, and much more.

What you will get from this handbook:

- A complete, structured learning path from zero to job-ready
- Clear explanations with real-life analogies — no jargon
- Hundreds of code examples you can run immediately
- 3 real-world projects to build your portfolio
- 175 interview questions with answers to crack your first job
- Cheat sheets and revision notes for quick review

How to use this book:

Read each chapter in order. Type every code example by hand — do not copy-paste. Doing the exercises at the end of each chapter is not optional; it is where real learning happens. Refer back to the cheat sheets during revision.

By the time you finish Chapter 13, you will have built three complete applications. By the time you finish Chapter 14, you will be ready to walk into a Python interview with confidence.

■ **Note:** All code in this handbook is compatible with Python 3.10 and above.

Let's begin your Python journey. It will change your career.

TABLE OF CONTENTS

Chapter 1	Introduction to Programming	Pg 5
Chapter 2	Python Fundamentals	Pg 8
Chapter 3	Operators and Decision Making	Pg 13
Chapter 4	Loops	Pg 17
Chapter 5	Functions	Pg 21
Chapter 6	Python Data Structures	Pg 25
Chapter 7	Strings	Pg 30
Chapter 8	Object-Oriented Programming	Pg 34
Chapter 9	File & Exception Handling	Pg 39
Chapter 10	Modules and Packages	Pg 42
Chapter 11	Python for Data Analysis	Pg 45
Chapter 12	Advanced Python Essentials	Pg 49
Chapter 13	Real-World Projects	Pg 53
Chapter 14	Interview Preparation (175 Q&A;)	Pg 64
Chapter 15	Quick Revision Guide & Cheat Sheets	Pg 75

CHAPTER 1

Introduction to Programming

Programming is the art of giving precise instructions to a computer so it can perform a task automatically. Think of it as writing a recipe — but for a machine that follows every step exactly as written, with no room for guessing.

1.1 What Is Programming?

A computer is powerful but dumb. It can perform billions of operations per second, but it does *exactly* what it is told — nothing more, nothing less. Programming is how we tell the computer what to do.

Real-Life Analogy: Think of a computer as a highly efficient chef who can cook at superhuman speed but needs every instruction spelled out. 'Add a pinch of salt' is too vague. You must say: 'Open the salt box. Pick up exactly 1/8 teaspoon. Tilt hand over pot. Release salt. Close salt box.'

A **program** is a set of such instructions, written in a language the computer understands.

1.2 Why Programming Exists

Humans are slow at repetitive tasks. Computers are fast — but they need instructions. Programming bridges the gap:

- Automation: Pay 10,000 employees on the same day without human error
- Speed: Analyse 1 million customer records in 2 seconds
- Scale: Serve 1 billion YouTube videos simultaneously
- Accuracy: Calculate rocket trajectories to 10 decimal places

1.3 How Software Works

Stage	What Happens
You write code	Source code in Python (text file)
Interpreter reads it	Python converts it to machine instructions
CPU executes it	Processor runs the instructions
Result appears	Output on screen, file saved, etc.

1.4 Why Python?

Python was created by Guido van Rossum and released in 1991. It is consistently ranked the #1 most popular language worldwide.

Advantage	Explanation
-----------	-------------

Readable syntax	Python reads almost like English — easy to learn
Huge community	Millions of tutorials, libraries, Stack Overflow answers
Versatile	Web, AI, data science, automation, games, scripts
High demand	Top 3 most-wanted skill in tech job listings globally
Free and open source	No licence cost; runs on Windows, Mac, Linux

■ **Tip:** Python is used by Google, Netflix, NASA, Instagram, Spotify, and Dropbox.

1.5 The Programming Mindset

Good programmers think in steps. When solving a problem, ask:

- What is the input?
- What should the output be?
- What steps transform the input into the output?
- What could go wrong, and how do we handle it?

This is called **algorithmic thinking**. Example: Making tea.

Step	Action
1	Boil water
2	Place teabag in cup
3	Pour boiling water
4	Wait 3 minutes
5	Remove teabag
6	Add milk/sugar as desired
7	Serve

Chapter 1 — Quick Exercises

Exercise 1: Write the steps (algorithm) to log into a website. Think about every small action.

Exercise 2: List 5 things you do every day that could be automated by a computer.

Exercise 3: Research one real-world product built with Python and write two sentences about it.

■ **Interview Q:** What is the difference between a compiled language and an interpreted language? Where does Python fit?

■ **Interview Q:** Name three industries where Python is heavily used.

CHAPTER 2

Python Fundamentals

2.1 Installing Python

- Go to **python.org** and download Python 3.11+
- Run the installer. On Windows: tick 'Add Python to PATH'
- Open Terminal (Mac/Linux) or Command Prompt (Windows)
- Type **python --version** and press Enter

```
python --version
# Expected output:
Python 3.11.4
```

■ **Tip:** Use VS Code as your code editor. It is free and has excellent Python support.

2.2 Your First Python Program

Create a file called **hello.py** and type:

```
print("Hello, World!")
```

Run it: **python hello.py**

```
Hello, World!
```

Congratulations — you are a programmer!

2.3 Variables

A **variable** is a named storage location in memory. Think of it as a labelled box.

```
name = "Alice" # string variable
age = 25 # integer variable
height = 5.6 # float variable
is_student = True # boolean variable
```

Rules for variable names:

- Start with a letter or underscore (`_`)
- No spaces — use underscore: `my_name`, not `my name`
- Case-sensitive: `age`, `Age`, `AGE` are three different variables
- No Python keywords: `if`, `for`, `while`, `class`, etc.

■ **Watch out:** Python uses dynamic typing — you do not declare the type; Python figures it out.

2.4 Data Types

Type	Example	Description
int	age = 25	Whole numbers

float	price = 9.99	Decimal numbers
str	name = "Alice"	Text (wrapped in quotes)
bool	active = True	True or False only
NoneType	result = None	Represents 'nothing'

```
x = 10
print(type(x)) # <class 'int'>
y = 3.14
print(type(y)) # <class 'float'>
z = "hello"
print(type(z)) # <class 'str'>
```

2.5 Input and Output

```
# Output
print("Welcome to Python!")
print("Sum:", 5 + 3)

# Input
name = input("Enter your name: ")
print("Hello,", name)
```

■ **Note:** input() always returns a string. Convert it if you need a number.

2.6 Type Conversion

```
# String to int
age_str = input("Enter age: ") # "25"
age_int = int(age_str) # 25

# String to float
price_str = "9.99"
price_flt = float(price_str)

# Any to string
num = 42
num_str = str(num) # "42"

# int to float
x = float(5) # 5.0
```

	Converts To	Example
	Integer	int('10') → 10
	Float	float('3.5') → 3.5
	String	str(99) → '99'
	Boolean	bool(0) → False

2.7 Comments

```
# This is a single-line comment

"""
This is a
multi-line comment (docstring)
"""

name = "Alice" # inline comment
```

■ **Tip:** Write comments to explain WHY, not WHAT. The code already shows what.

2.8 Constants

Python has no built-in constant keyword, but the convention is ALL_CAPS:

```
PI = 3.14159
MAX_SIZE = 100
APP_NAME = "MyApp"
```

Chapter 2 — Practice Programs

```
# Program 1: Personal Info Card
name = input("Name: ")
age = int(input("Age: "))
city = input("City: ")
print("--- Your Profile ---")
print(f"Name : {name}")
print(f"Age : {age}")
print(f"City : {city}")
```

```
# Program 2: Simple Calculator
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
print("Sum:", a + b)
print("Difference:", a - b)
print("Product:", a * b)
print("Quotient:", a / b)
```

■ **Interview Q:** What is the difference between int and float in Python?

■ **Interview Q:** What does the input() function return? How would you get an integer from the user?

■ **Interview Q:** Can a variable name start with a number in Python? Give an example.

CHAPTER 3

Operators and Decision Making

3.1 Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
-	Subtraction	10 - 4	6
*	Multiplication	3 * 4	12
/	Division (float)	7 / 2	3.5
//	Floor Division	7 // 2	3
%	Modulus (remainder)	7 % 2	1
**	Exponentiation	2 ** 8	256

```
a, b = 17, 5
print(a + b) # 22
print(a // b) # 3 (floor division)
print(a % b) # 2 (remainder)
print(a ** 2) # 289
```

3.2 Comparison Operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	7 > 3	True
<	Less than	2 < 9	True
>=	Greater than or equal	5 >= 5	True
<=	Less than or equal	4 <= 3	False

3.3 Logical Operators

Operator	Meaning	Example	Result
and	Both must be True	5>2 and 3>1	True
or	At least one must be True	5>10 or 3>1	True
not	Reverses the result	not(5>10)	True

3.4 Decision Making: if / elif / else

Python uses indentation (4 spaces) to define code blocks — no curly braces.

```
score = 75

if score >= 90:
    print("Grade: A")
elif score >= 75:
    print("Grade: B")
elif score >= 60:
    print("Grade: C")
else:
    print("Grade: F")

# Output: Grade: B
```

3.5 Nested Conditions

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("Show ID to enter")
else:
    print("You must be 18 or older")
```

3.6 Ternary (One-Line If)

```
# Syntax: value_if_true if condition else value_if_false
x = 10
result = "Even" if x % 2 == 0 else "Odd"
print(result) # Even
```

Chapter 3 — Real-World Exercise

```
# BMI Calculator
weight = float(input("Weight in kg: "))
height = float(input("Height in m: "))
bmi = weight / (height ** 2)
print(f"BMI: {bmi:.2f}")
if bmi < 18.5:
    print("Underweight")
elif bmi < 25:
    print("Normal")
elif bmi < 30:
    print("Overweight")
else:
    print("Obese")
```

■ **Interview Q:** What is the difference between = and == in Python?

■ **Interview Q:** What is the output of: `print(10 % 3)`?

■ **Interview Q:** Explain the 'and' and 'or' operators with examples.

CHAPTER 4

Loops

A loop repeats a block of code multiple times. Without loops, printing 1 to 1,000,000 would require a million `print()` statements!

4.1 The for Loop

```
# Loop over a range
for i in range(5):
    print(i) # Prints 0 1 2 3 4

# range(start, stop, step)
for i in range(1, 11, 2):
    print(i) # 1 3 5 7 9

# Loop over a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

■ **Note:** `range(5)` gives 0,1,2,3,4 — the stop value is excluded.

4.2 The while Loop

```
# Countdown
counter = 5
while counter > 0:
    print(counter)
    counter -= 1
    print("Blastoff!")

# User input loop
password = ""
while password != "secret123":
    password = input("Enter password: ")
    print("Access granted!")
```

■■ **Watch out:** A while loop without a proper exit condition runs forever (infinite loop). Always ensure the condition eventually becomes `False`.

4.3 break and continue

```
# break – exit the loop immediately
for i in range(10):
    if i == 5:
        break
    print(i) # Prints 0 1 2 3 4

# continue – skip current iteration
for i in range(10):
```

```
if i % 2 == 0:
    continue
print(i) # Prints 1 3 5 7 9 (odd numbers only)
```

4.4 Nested Loops

```
# Multiplication table
for i in range(1, 6):
    for j in range(1, 6):
        print(f"{i*j:4}", end="")
    print() # newline after each row

# Output:
1 2 3 4 5
2 4 6 8 10
3 6 9 12 15
4 8 12 16 20
5 10 15 20 25
```

4.5 Common Loop Patterns

```
# Summing numbers
total = 0
for n in range(1, 101):
    total += n
print("Sum 1 to 100:", total) # 5050

# Counting specific items
nums = [4, 7, 2, 9, 1, 8, 3, 6]
count_above_5 = 0
for n in nums:
    if n > 5:
        count_above_5 += 1
print("Above 5:", count_above_5) # 4

# Finding maximum
max_val = nums[0]
for n in nums:
    if n > max_val:
        max_val = n
print("Max:", max_val) # 9
```

Chapter 4 — Practice Programs

```
# Fibonacci sequence up to n terms
n = int(input("How many terms? "))
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

```
# Number guessing game
import random
```

```
secret = random.randint(1, 100)
guess = 0
while guess != secret:
    guess = int(input("Guess (1-100): "))
    if guess < secret:
        print("Too low!")
    elif guess > secret:
        print("Too high!")
    print("Correct! You win!")
```

■ **Interview Q:** What is the difference between break and continue?

■ **Interview Q:** How many times does this loop run: for i in range(2, 10, 3)?

■ **Interview Q:** Write a loop to print all even numbers from 1 to 50.

CHAPTER 5

Functions

A function is a reusable block of code that performs a specific task. Think of it as a named recipe: you define it once, then call it whenever you need it.

5.1 Defining and Calling Functions

```
def greet():  
    print("Hello! Welcome to Python.")  
  
greet() # Call the function  
greet() # Call it again – reusable!
```

5.2 Parameters and Arguments

```
# Parameters are placeholders in the definition  
def greet(name, age):  
    print(f"Hello {name}, you are {age} years old.")  
  
# Arguments are actual values passed when calling  
greet("Alice", 30)  
greet("Bob", 25)
```

Default parameters:

```
def power(base, exp=2): # exp defaults to 2  
    return base ** exp  
  
print(power(3)) # 9 (3 squared)  
print(power(3, 3)) # 27 (3 cubed)
```

5.3 Return Values

```
def add(a, b):  
    return a + b  
  
result = add(5, 3)  
print(result) # 8  
  
# Multiple return values  
def min_max(numbers):  
    return min(numbers), max(numbers)  
  
lo, hi = min_max([4, 1, 9, 2, 7])  
print(lo, hi) # 1 9
```

5.4 Variable Scope

Local scope: variable defined inside a function — only accessible there.

Global scope: variable defined at the top level — accessible everywhere.

```
x = 10 # global

def show():
    y = 20 # local
    print(x) # OK – can read global
    print(y) # OK

show()
print(x) # OK
# print(y) # ERROR – y is local to show()

# Modifying a global variable inside a function
counter = 0

def increment():
    global counter
    counter += 1

increment()
increment()
print(counter) # 2
```

5.5 *args and **kwargs

```
# *args – variable number of positional arguments
def total(*args):
    return sum(args)

print(total(1, 2, 3)) # 6
print(total(10, 20, 30, 40)) # 100

# **kwargs – variable keyword arguments
def profile(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

profile(name="Alice", age=30, city="London")
```

5.6 Lambda Functions

A lambda is a small anonymous function written in one line.

```
# Regular function
def square(x):
    return x ** 2

# Equivalent lambda
square = lambda x: x ** 2
print(square(5)) # 25

# Lambda with multiple arguments
add = lambda a, b: a + b
print(add(3, 7)) # 10

# Common use: sorting
```



```
students = [("Alice", 85), ("Bob", 92), ("Carol", 78)]
students.sort(key=lambda s: s[1], reverse=True)
print(students) # [(Bob,92),(Alice,85),(Carol,78)]
```

5.7 Recursion

A function that calls itself is recursive. Every recursive function needs a base case to stop.

```
def factorial(n):
    if n == 0: # base case
        return 1
    return n * factorial(n - 1) # recursive case

print(factorial(5)) # 120 (5*4*3*2*1)
```

Chapter 5 — Exercises

```
# Exercise: Temperature converter
def celsius_to_fahrenheit(c):
    return (c * 9/5) + 32

def fahrenheit_to_celsius(f):
    return (f - 32) * 5/9

temp = float(input("Enter Celsius: "))
print(f"{temp}°C = {celsius_to_fahrenheit(temp):.1f}°F")
```

■ **Interview Q:** What is the difference between a parameter and an argument?

■ **Interview Q:** What is the output of a function that has no return statement?

■ **Interview Q:** Explain what *args does with an example.

■ **Interview Q:** What is a lambda function? When would you use one?

CHAPTER 6

Python Data Structures

Python has four built-in data structures: Lists, Tuples, Sets, and Dictionaries. Choosing the right one is crucial for efficient code.

6.1 Comparison Overview

Feature	List	Tuple	Set	Dictionary
Syntax	[]	()	{ }	{ key:val }
Ordered	Yes	Yes	No	Yes (3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates	Yes	Yes	No	Keys: No
Use case	General collection	Immutable data	Unique items	Key-value pairs

6.2 Lists

The most versatile data structure. Like a shopping list — ordered and changeable.

```
fruits = ["apple", "banana", "cherry"]

# Indexing (0-based)
print(fruits[0]) # apple
print(fruits[-1]) # cherry (last item)

# Slicing
print(fruits[0:2]) # ['apple', 'banana']

# Common methods
fruits.append("mango") # add to end
fruits.insert(1, "kiwi") # insert at index 1
fruits.remove("banana") # remove by value
fruits.pop() # remove last
fruits.sort() # sort in place
print(len(fruits)) # count items
print("mango" in fruits) # True/False membership
```

6.3 Tuples

Immutable (unchangeable) lists. Use them for data that must not be modified.

```
coordinates = (28.6139, 77.2090) # Delhi GPS
rgb_red = (255, 0, 0)

# Access same as list
print(coordinates[0]) # 28.6139
```

```
# Unpacking
lat, lon = coordinates
print(lat, lon)

# Tuple methods
t = (1, 2, 2, 3, 2)
print(t.count(2)) # 3
print(t.index(3)) # 3
```

■ **Tip:** Tuples are faster than lists. Use tuples when data should not change: coordinates, RGB values, database records.

6.4 Sets

An unordered collection of unique items. Perfect for membership testing and removing duplicates.

```
s1 = {1, 2, 3, 4, 5}
s2 = {3, 4, 5, 6, 7}

# Set operations (like Venn diagrams)
print(s1 | s2) # Union: {1,2,3,4,5,6,7}
print(s1 & s2) # Intersection: {3,4,5}
print(s1 - s2) # Difference: {1,2}

# Remove duplicates from a list
data = [1, 2, 2, 3, 3, 3, 4]
unique = list(set(data))
print(unique) # [1, 2, 3, 4] (order may vary)

# Membership (very fast)
print(3 in s1) # True
```

6.5 Dictionaries

Key-value pairs. Like a real dictionary where you look up a word (key) to find its meaning (value).

```
student = {
    "name": "Alice",
    "age": 21,
    "grade": "A",
    "course": "Computer Science"
}

# Access
print(student["name"]) # Alice
print(student.get("gpa", 0.0)) # 0.0 (safe access)

# Modify
student["grade"] = "A+"
student["year"] = 2

# Common methods
print(student.keys()) # all keys
print(student.values()) # all values
```

```
print(student.items()) # key-value pairs

# Loop through
for key, value in student.items():
    print(f"{key}: {value}")
```

6.6 Nested Data Structures

```
# List of dictionaries (very common in real apps)
students = [
    {"name": "Alice", "score": 92},
    {"name": "Bob", "score": 85},
    {"name": "Carol", "score": 97},
]

for s in students:
    print(f"{s['name']}: {s['score']}")

# Find highest score
best = max(students, key=lambda s: s["score"])
print("Top student:", best["name"])
```

Chapter 6 — Exercises

```
# 1. Find all unique words in a sentence
sentence = "the cat sat on the mat the cat sat"
words = sentence.split()
unique_words = set(words)
print(f"Unique words: {len(unique_words)}")
print(unique_words)

# 2. Word frequency counter
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1
print(freq)
```

■ **Interview Q:** What is the difference between a list and a tuple?

■ **Interview Q:** Can a dictionary have duplicate keys?

■ **Interview Q:** How do you remove duplicate elements from a list? What data structure helps?

■ **Interview Q:** What does `dict.get(key, default)` do differently from `dict[key]`?

CHAPTER 7

Strings

Strings are sequences of characters. Text processing is one of the most common programming tasks, and Python has exceptional built-in string support.

7.1 String Basics

```
s = "Hello, World!"

print(s[0]) # H (indexing)
print(s[-1]) # ! (last char)
print(s[0:5]) # Hello (slicing)
print(s[::2]) # Hlo ol! (step slicing)
print(len(s)) # 13 (length)
print(s.upper()) # HELLO, WORLD!
print(s.lower()) # hello, world!
```

7.2 String Methods

Method	What It Does	Example
<code>.strip()</code>	Remove whitespace at ends	" hi ".strip() → "hi"
<code>.split(sep)</code>	Split into a list	"a,b,c".split(",") → ["a","b","c"]
<code>.join(iterable)</code>	Join list into string	",".join(["a","b"]) → "a,b"
<code>.replace(a,b)</code>	Replace substring	"hello".replace("l","r") → "herro"
<code>.find(s)</code>	Find index or -1	"hello".find("ll") → 2
<code>.startswith(s)</code>	Check prefix	"hello".startswith("he") → True
<code>.endswith(s)</code>	Check suffix	"hello".endswith("lo") → True
<code>.isdigit()</code>	Check all digits	"123".isdigit() → True
<code>.count(s)</code>	Count occurrences	"hello".count("l") → 2

7.3 String Formatting

```
name = "Alice"
age = 30
price = 9.99

# f-string (recommended, Python 3.6+)
print(f"Name: {name}, Age: {age}")
print(f"Price: ${price:.2f}")
print(f"Square of 7: {7**2}")

# format() method
```

```
print("Name: {}, Age: {}".format(name, age))

# % formatting (old style)
print("Name: %s, Age: %d" % (name, age))
```

7.4 String Manipulation Examples

```
# Reverse a string
s = "Python"
reversed_s = s[::-1]
print(reversed_s) # nohtyP

# Check palindrome
def is_palindrome(s):
    s = s.lower().replace(" ", "")
    return s == s[::-1]

print(is_palindrome("racecar")) # True
print(is_palindrome("A man a plan a canal Panama")) # True

# Count vowels
def count_vowels(s):
    return sum(1 for c in s.lower() if c in "aeiou")

print(count_vowels("Hello World")) # 3

# Title case
print("hello world".title()) # Hello World
```

7.5 Common Interview String Problems

```
# 1. Check anagram
def is_anagram(s1, s2):
    return sorted(s1.lower()) == sorted(s2.lower())
print(is_anagram("listen", "silent")) # True

# 2. Count words
sentence = "The quick brown fox jumps"
words = sentence.split()
print(len(words)) # 5

# 3. Remove duplicates preserving order
def unique_chars(s):
    seen = set()
    return "".join(c for c in s if not (c in seen or seen.add(c)))
print(unique_chars("programming")) # progamin

# 4. Caesar cipher
def caesar(text, shift):
    result = ""
    for c in text:
        if c.isalpha():
            base = ord('A') if c.isupper() else ord('a')
```

```
result += chr((ord(c) - base + shift) % 26 + base)
else:
    result += c
return result
print(caesar("Hello", 3)) # Khoor
```

- **Interview Q:** How do you reverse a string in Python?
- **Interview Q:** What is the difference between `.find()` and `.index()`?
- **Interview Q:** How do you check if a string is a palindrome?
- **Interview Q:** What does `split()` return when called with no arguments?

CHAPTER 8

Object-Oriented Programming (OOP)

OOP organises code around objects — self-contained units that combine data (attributes) and behaviour (methods). Think of a **class** as a blueprint for a house, and an **object** as a house built from that blueprint.

8.1 Classes and Objects

```
class Dog:
    # Class attribute (shared by all instances)
    species = "Canis lupus familiaris"

    # Constructor: called when object is created
    def __init__(self, name, age):
        self.name = name # instance attribute
        self.age = age

    # Method
    def bark(self):
        print(f"{self.name} says: Woof!")

    def info(self):
        print(f"{self.name} is {self.age} years old")

# Create objects
dog1 = Dog("Buddy", 3)
dog2 = Dog("Max", 5)

dog1.bark() # Buddy says: Woof!
dog2.info() # Max is 5 years old
print(Dog.species) # Canis lupus familiaris
```

8.2 Encapsulation

Encapsulation hides internal data and exposes only what's needed. Private attributes start with `_` or `__`.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance # private

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited {amount}. Balance: {self.__balance}")

    def withdraw(self, amount):
        if amount > self.__balance:
            print("Insufficient funds")
        else:
            self.__balance -= amount
```



```
def get_balance(self):
    return self.__balance # controlled access

acc = BankAccount("Alice", 1000)
acc.deposit(500)
print(acc.get_balance()) # 1500
# print(acc.__balance) # AttributeError – protected!
```

8.3 Inheritance

Inheritance lets a child class acquire attributes and methods from a parent class. Analogy: A Smartphone inherits features of a Phone and adds new ones.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound")

class Cat(Animal): # Cat inherits from Animal
    def speak(self): # override parent method
        print(f"{self.name} says: Meow!")

class Dog(Animal):
    def speak(self):
        print(f"{self.name} says: Woof!")

animals = [Cat("Whiskers"), Dog("Buddy"), Animal("Generic")]
for a in animals:
    a.speak() # Polymorphism!
```

8.4 Polymorphism

Polymorphism means 'many forms'. The same method name behaves differently depending on the object. The loop above demonstrates polymorphism — each `animal.speak()` does something different.

8.5 Special (Dunder) Methods

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __str__(self): # print(point)
        return f"Point({self.x}, {self.y})"

    def __add__(self, other): # point1 + point2
        return Point(self.x+other.x, self.y+other.y)

    def __eq__(self, other): # point1 == point2
        return self.x==other.x and self.y==other.y
```

```
p1 = Point(1, 2)
p2 = Point(3, 4)
print(p1) # Point(1, 2)
print(p1 + p2) # Point(4, 6)
print(p1 == p2) # False
```

Mini Project: Library System

```
class Book:
    def __init__(self, title, author, copies):
        self.title = title
        self.author = author
        self.copies = copies

    def __str__(self):
        return f'"{self.title}" by {self.author} ({self.copies} copies)'

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)
        print(f"Added: {book.title}")

    def borrow(self, title):
        for book in self.books:
            if book.title == title:
                if book.copies > 0:
                    book.copies -= 1
                    print(f"Borrowed: {title}")
                else:
                    print("No copies available")
                    return
        print("Book not found")

    def list_books(self):
        for book in self.books:
            print(book)

lib = Library()
lib.add_book(Book("Python Crash Course", "Eric Matthes", 3))
lib.add_book(Book("Clean Code", "Robert Martin", 2))
lib.list_books()
lib.borrow("Clean Code")
lib.list_books()
```

■ **Interview Q:** What is the difference between a class and an object?

■ **Interview Q:** What does `__init__` do?

■ **Interview Q:** Explain inheritance with a real-world example.

■ **Interview Q:** What is the difference between method overriding and method overloading?

CHAPTER 9

File Handling and Exception Handling

9.1 File Handling

Mode	Description
'r'	Read (default). File must exist.
'w'	Write. Creates file; overwrites if exists.
'a'	Append. Adds to end of file.
'r+'	Read and write.
'rb'	Read binary (images, PDFs, etc.)

```
# Writing to a file
with open("notes.txt", "w") as f:
    f.write("Hello from Python!\n")
    f.write("Second line.\n")

# Reading entire file
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)

# Reading line by line (memory efficient)
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())

# Appending
with open("notes.txt", "a") as f:
    f.write("Appended line.\n")
```

■ **Tip:** Always use 'with open(...)' — it automatically closes the file even if an error occurs.

9.2 Working with CSV Files

```
import csv

# Write CSV
students = [
    ["Alice", 90], ["Bob", 85], ["Carol", 92]
]
with open("students.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Score"]) # header
    writer.writerows(students)

# Read CSV
with open("students.csv", "r") as f:
```

```
reader = csv.DictReader(f)
for row in reader:
    print(f"{row['Name']}: {row['Score']}")
```

9.3 Exception Handling

Exceptions are runtime errors. Handling them prevents your program from crashing.

Exception	When It Occurs
ValueError	Invalid value: int('abc')
ZeroDivisionError	Divide by zero: 10/0
FileNotFoundError	File doesn't exist
TypeError	Wrong type: 'a' + 1
IndexError	List index out of range
KeyError	Dict key not found
AttributeError	Object has no such attribute

```
# Basic try/except
try:
    x = int(input("Enter a number: "))
    result = 100 / x
    print(f"Result: {result}")
except ValueError:
    print("That's not a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:
    print(f"Unexpected error: {e}")
else:
    print("No errors occurred!") # runs if no exception
finally:
    print("This ALWAYS runs") # cleanup code here
```

9.4 Raising Exceptions

```
def set_age(age):
    if age < 0 or age > 150:
        raise ValueError(f"Invalid age: {age}")
    return age

try:
    set_age(-5)
except ValueError as e:
    print(e) # Invalid age: -5
```

9.5 Custom Exceptions

```
class InsufficientFundsError(Exception):
    def __init__(self, amount, balance):
        self.message = f"Cannot withdraw {amount}. Balance: {balance}"
        super().__init__(self.message)

    def withdraw(amount, balance):
        if amount > balance:
            raise InsufficientFundsError(amount, balance)
        return balance - amount

try:
    new_bal = withdraw(500, 200)
except InsufficientFundsError as e:
    print(e)
```

■ **Interview Q:** What is the purpose of the finally block?

■ **Interview Q:** What is the difference between a syntax error and an exception?

■ **Interview Q:** How do you create a custom exception?

CHAPTER 10

Modules and Packages

A module is a .py file containing reusable code. A package is a folder of modules. Python ships with hundreds of built-in modules, and thousands more are available via pip.

10.1 Importing Modules

```
import math
import random
import datetime

# math module
print(math.pi) # 3.14159...
print(math.sqrt(144)) # 12.0
print(math.ceil(4.2)) # 5
print(math.floor(4.9)) # 4

# random module
print(random.randint(1, 100)) # random int 1-100
print(random.choice(["rock", "paper", "scissors"]))
print(random.random()) # float 0.0-1.0

# datetime module
now = datetime.datetime.now()
print(now.strftime("%d/%m/%Y %H:%M"))

# Import specific names
from math import sqrt, pi
print(sqrt(81)) # 9.0 – no 'math.' prefix needed

# Alias
import numpy as np
import pandas as pd
```

10.2 Creating Your Own Module

```
# File: mymath.py
def add(a, b):
    return a + b

def factorial(n):
    return 1 if n == 0 else n * factorial(n-1)

PI = 3.14159

# File: main.py
import mymath
print(mymath.add(5, 3)) # 8
print(mymath.factorial(5)) # 120
print(mymath.PI) # 3.14159
```

10.3 Essential Standard Library Modules

Module	Purpose	Example Use
os	OS interaction	os.getcwd(), os.listdir()
sys	Python runtime info	sys.argv, sys.exit()
json	JSON encode/decode	json.loads(), json.dumps()
re	Regular expressions	re.findall(), re.match()
collections	Specialised containers	Counter, defaultdict, deque
itertools	Iterator building blocks	chain, combinations, product
datetime	Date and time	datetime.now(), timedelta
pathlib	Filesystem paths	Path.exists(), Path.mkdir()
time	Time functions	time.sleep(), time.time()

10.4 pip — Installing Third-Party Packages

```
# Install a package
pip install requests
pip install pandas
pip install numpy

# Install specific version
pip install requests==2.31.0

# List installed packages
pip list

# Save requirements
pip freeze > requirements.txt

# Install from requirements
pip install -r requirements.txt
```

10.5 Virtual Environments

```
# Create virtual environment
python -m venv myenv

# Activate (Windows)
myenv\Scripts\activate

# Activate (Mac/Linux)
source myenv/bin/activate

# Deactivate
deactivate
```

■ **Tip:** Always create a virtual environment for each project. This keeps dependencies isolated.

10.6 JSON Module — Real-World Use

```
import json

# Python dict to JSON string
student = {"name": "Alice", "age": 21, "scores": [90, 85, 92]}
json_str = json.dumps(student, indent=2)
print(json_str)

# JSON string to Python dict
json_data = '{"city": "London", "pop": 8900000}'
data = json.loads(json_data)
print(data["city"]) # London

# Read/write JSON files
with open("data.json", "w") as f:
    json.dump(student, f, indent=2)

with open("data.json", "r") as f:
    loaded = json.load(f)
```

■ **Interview Q:** What is the difference between a module and a package?

■ **Interview Q:** How do you install a third-party library in Python?

■ **Interview Q:** What is a virtual environment and why is it used?

CHAPTER 11

Introduction to Python for Data Analysis

Python is the dominant language for data analysis. NumPy provides fast numerical computing; Pandas provides powerful data manipulation. Together they form the foundation of all data science work.

11.1 NumPy Basics

```
import numpy as np

# Create arrays
arr = np.array([1, 2, 3, 4, 5])
matrix = np.array([[1, 2, 3],
[4, 5, 6]])

print(arr.shape) # (5,)
print(matrix.shape) # (2, 3)
print(arr.dtype) # int64

# Array operations (element-wise, very fast)
print(arr * 2) # [ 2 4 6 8 10]
print(arr + arr) # [ 2 4 6 8 10]
print(arr ** 2) # [ 1 4 9 16 25]

# Statistical functions
print(np.mean(arr)) # 3.0
print(np.sum(arr)) # 15
print(np.max(arr)) # 5
print(np.std(arr)) # 1.41...

# Useful array creation
zeros = np.zeros((3, 3))
ones = np.ones((2, 4))
rng = np.arange(0, 10, 2) # [0 2 4 6 8]
rand = np.random.rand(3, 3) # 3x3 random floats

# Indexing and slicing
data = np.array([10, 20, 30, 40, 50])
print(data[1:4]) # [20 30 40]
print(data[data > 25]) # [30 40 50] – boolean indexing
```

11.2 Pandas Basics

```
import pandas as pd

# Series (one-dimensional)
ages = pd.Series([25, 30, 22, 28], index=["Alice", "Bob", "Carol", "Dan"])
print(ages["Alice"]) # 25
print(ages.mean()) # 26.25
```

```
# DataFrame (two-dimensional table)
df = pd.DataFrame({
    "Name": ["Alice", "Bob", "Carol"],
    "Age": [25, 30, 22],
    "Score": [92, 85, 97],
    "Passed": [True, True, True],
})
print(df.head()) # first 5 rows
print(df.info()) # column types and nulls
print(df.describe()) # statistics
```

11.3 DataFrame Operations

```
# Select columns
print(df["Name"]) # one column
print(df[["Name", "Score"]]) # multiple columns

# Filter rows
high_scorers = df[df["Score"] >= 90]
print(high_scorers)

# Add a column
df["Grade"] = df["Score"].apply(
    lambda s: "A" if s >= 90 else "B"
)

# Sort
df_sorted = df.sort_values("Score", ascending=False)

# Aggregate
print(df.groupby("Passed")["Score"].mean())
```

11.4 Reading CSV Files

```
# Read
df = pd.read_csv("sales.csv")

# Quick exploration
print(df.shape) # (rows, cols)
print(df.columns) # column names
print(df.head(10)) # first 10 rows
print(df.isnull().sum()) # count missing values

# Handle missing values
df.dropna(inplace=True) # drop rows with NaN
df["Age"].fillna(df["Age"].mean(), inplace=True) # fill with mean

# Write back
df.to_csv("clean_sales.csv", index=False)
```

■ **Tip:** Install: `pip install pandas numpy`. These are the two most important data science libraries.

■ **Interview Q:** What is the difference between a NumPy array and a Python list?

■ **Interview Q:** What is a DataFrame?

■ **Interview Q:** How do you read a CSV file into Pandas?

CHAPTER 12

Advanced Python Essentials

12.1 List Comprehensions

A concise way to create lists. More readable and faster than a for loop + append.

```
# Regular loop
squares = []
for n in range(1, 11):
    squares.append(n ** 2)

# List comprehension (same result)
squares = [n ** 2 for n in range(1, 11)]
print(squares) # [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

# With condition (filter)
evens = [n for n in range(20) if n % 2 == 0]
print(evens) # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Nested comprehension
matrix = [[i*j for j in range(1,4)] for i in range(1,4)]
print(matrix) # [[1,2,3],[2,4,6],[3,6,9]]

# Dictionary comprehension
word_len = {word: len(word) for word in ["apple","banana","cherry"]}
print(word_len) # {'apple': 5, 'banana': 6, 'cherry': 6}

# Set comprehension
unique_sq = {n**2 for n in range(-5, 6)}
print(unique_sq) # {0, 1, 4, 9, 16, 25}
```

12.2 Generators

Generators produce values on demand (lazy evaluation). They use far less memory than lists for large datasets.

```
# Generator function (uses yield instead of return)
def count_up(limit):
    n = 0
    while n < limit:
        yield n # pauses here and returns n
        n += 1 # resumes from here next time

gen = count_up(5)
print(next(gen)) # 0
print(next(gen)) # 1
for val in gen: # continue iteration
    print(val) # 2, 3, 4

# Generator expression (like list comprehension with ())
```

```
big_squares = (n**2 for n in range(10_000_000))
print(next(big_squares)) # 0 – only computed on demand!

# Real use: process huge files line by line
def read_large_file(path):
    with open(path) as f:
        for line in f:
            yield line.strip()
```

■ **Tip:** Use generators when processing millions of items — they use $O(1)$ memory instead of $O(n)$.

12.3 Iterators

```
# Any object with __iter__() and __next__() is an iterator
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        val = self.current
        self.current -= 1
        return val

for n in Countdown(5):
    print(n) # 5 4 3 2 1
```

12.4 Decorators

A decorator wraps a function to add behaviour before or after it runs — without changing the original code.

```
import time
import functools

# Decorator definition
def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs) # run original function
        end = time.time()
        print(f"{func.__name__} took {end-start:.4f}s")
        return result
    return wrapper

# Apply decorator with @
@timer
def slow_sum(n):
    return sum(range(n))
```

```
slow_sum(1_000_000) # prints time taken

# Logger decorator
def logger(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} with {args}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} returned {result}")
        return result
    return wrapper

@logger
def add(a, b):
    return a + b

add(3, 7)
```

12.5 Context Managers

```
# Built-in: with open
with open("file.txt") as f:
    data = f.read() # file closed automatically

# Custom context manager
from contextlib import contextmanager

@contextmanager
def timer_ctx(label):
    import time
    start = time.time()
    yield
    elapsed = time.time() - start
    print(f"{label}: {elapsed:.3f}s")

with timer_ctx("Data processing"):
    total = sum(range(1_000_000))
```

■ **Interview Q:** What is the difference between a generator and a list?

■ **Interview Q:** What does the yield keyword do?

■ **Interview Q:** Explain decorators in simple terms with an example.

■ **Interview Q:** What is lazy evaluation?

CHAPTER 13

Real-World Projects

Building real projects is the fastest way to solidify your skills. Each project here combines multiple concepts from earlier chapters.

PROJECT 1: Student Management System

Requirements: Add students, view all, search by name, update grades, delete student, save to JSON file.

```
# student_management.py
import json
import os

DATA_FILE = "students.json"

def load_students():
    if os.path.exists(DATA_FILE):
        with open(DATA_FILE) as f:
            return json.load(f)
    return []

def save_students(students):
    with open(DATA_FILE, "w") as f:
        json.dump(students, f, indent=2)

def add_student(students):
    name = input("Student name: ").strip()
    roll = input("Roll number: ").strip()
    grade = input("Grade (A/B/C/D/F): ").upper().strip()
    score = float(input("Score: "))
    students.append({"name": name, "roll": roll,
                    "grade": grade, "score": score})
    save_students(students)
    print(f"{name} added successfully.")

def view_all(students):
    if not students:
        print("No students found.")
        return
    print(f"\n{'No':<5}{'Name':<20}{'Roll':<12}{'Grade':<8}{'Score':<8}")
    print("-" * 55)
    for i, s in enumerate(students, 1):
        print(f"{i:<5}{s['name']:<20}{s['roll']:<12}{s['grade']:<8}{s['score']:<8}")

def search_student(students):
    name = input("Enter name to search: ").lower()
    found = [s for s in students if name in s["name"].lower()]
    if found:
        view_all(found)
    else:
        print("No matching student found.")
```

```

def update_grade(students):
    roll = input("Enter roll number: ").strip()
    for s in students:
        if s["roll"] == roll:
            s["grade"] = input("New grade: ").upper()
            s["score"] = float(input("New score: "))
            save_students(students)
            print("Updated!")
            return
    print("Student not found.")

def delete_student(students):
    roll = input("Roll number to delete: ").strip()
    orig_len = len(students)
    students[:] = [s for s in students if s["roll"] != roll]
    if len(students) < orig_len:
        save_students(students)
        print("Deleted successfully.")
    else:
        print("Student not found.")

def main():
    students = load_students()
    menu = {
        "1": ("Add Student", lambda: add_student(students)),
        "2": ("View All", lambda: view_all(students)),
        "3": ("Search Student", lambda: search_student(students)),
        "4": ("Update Grade", lambda: update_grade(students)),
        "5": ("Delete Student", lambda: delete_student(students)),
        "6": ("Exit", None),
    }
    while True:
        print("\n=== Student Management System ===")
        for key, (label, _) in menu.items():
            print(f" {key}. {label}")
        choice = input("Choice: ").strip()
        if choice == "6":
            print("Goodbye!")
            break
        elif choice in menu:
            menu[choice][1]()
        else:
            print("Invalid choice.")

if __name__ == "__main__":
    main()

```

PROJECT 2: Expense Tracker

Requirements: Add expense, view by category, monthly summary, set budget warnings.

```

# expense_tracker.py
import json, os

```



```

from datetime import datetime
from collections import defaultdict

FILE = "expenses.json"

def load(): return json.load(open(FILE)) if os.path.exists(FILE) else []
def save(data): json.dump(data, open(FILE, "w"), indent=2)

def add_expense(expenses):
    amount = float(input("Amount: "))
    category = input("Category (food/travel/bills/other): ").lower()
    desc = input("Description: ")
    expenses.append({
        "amount": amount,
        "category": category,
        "desc": desc,
        "date": datetime.now().strftime("%Y-%m-%d"),
    })
    save(expenses)
    print(f"Expense of {amount} added.")

def monthly_summary(expenses):
    month = input("Month (YYYY-MM): ")
    monthly = [e for e in expenses if e["date"].startswith(month)]
    if not monthly:
        print("No expenses for that month.")
        return
    by_cat = defaultdict(float)
    total = 0
    for e in monthly:
        by_cat[e["category"]] += e["amount"]
    total += e["amount"]
    print(f"\nSummary for {month}:")
    for cat, amt in sorted(by_cat.items()):
        print(f" {cat:15} : {amt:>8.2f}")
    print(f"{'TOTAL':15} : {total:>8.2f}")

def check_budget(expenses, budget=5000):
    month = datetime.now().strftime("%Y-%m")
    total = sum(e["amount"] for e in expenses
                if e["date"].startswith(month))
    print(f"\nThis month: {total:.2f} / {budget}")
    if total > budget:
        print("WARNING: You have exceeded your budget!")
    elif total > budget * 0.8:
        print("Alert: You have used 80% of your budget.")
    else:
        remaining = budget - total
        print(f"Remaining budget: {remaining:.2f}")

def main():
    expenses = load()
    while True:
        print("\n=== Expense Tracker ===")

```

```

print("1. Add Expense 2. Monthly Summary")
print("3. Budget Check 4. Exit")
choice = input("Choice: ")
if choice=="1": add_expense(expenses)
elif choice=="2": monthly_summary(expenses)
elif choice=="3": check_budget(expenses)
elif choice=="4": break

if __name__ == "__main__":
    main()

```

PROJECT 3: Inventory Management System

```

# inventory.py
import json, os
from datetime import datetime

FILE = "inventory.json"

def load(): return json.load(open(FILE)) if os.path.exists(FILE) else {}
def save(inv): json.dump(inv, open(FILE, "w"), indent=2)

def add_item(inv):
    code = input("Item code: ").upper()
    name = input("Item name: ")
    qty = int(input("Quantity: "))
    price = float(input("Unit price: "))
    if code in inv:
        inv[code]["quantity"] += qty
        print("Stock updated.")
    else:
        inv[code] = {"name": name, "quantity": qty,
                    "price": price, "sales": 0}
        print("Item added.")
        save(inv)

def sell_item(inv):
    code = input("Item code: ").upper()
    if code not in inv:
        print("Item not found"); return
    item = inv[code]
    qty = int(input(f"Qty to sell (in stock: {item['quantity']}): "))
    if qty > item["quantity"]:
        print("Insufficient stock"); return
    item["quantity"] -= qty
    item["sales"] += qty
    revenue = qty * item["price"]
    print(f"Sold! Revenue: {revenue:.2f}")
    save(inv)

def low_stock_alert(inv, threshold=5):
    print("\n--- Low Stock Alert ---")
    low = [(c, i) for c, i in inv.items() if i["quantity"] <= threshold]
    if not low:

```

```
print("All items sufficiently stocked.")
for code, item in low:
    print(f"{code}: {item['name']} - {item['quantity']} left")

def view_inventory(inv):
    print(f"\n{'Code':<8}{'Name':<20}{'Qty':>6}{'Price':>10}{'Sales':>8}")
    print("-" * 56)
    total_val = 0
    for code, item in inv.items():
        val = item["quantity"] * item["price"]
        total_val += val
    print(f"{code:<8}{item['name']:<20}{item['quantity']:>6}"
          f"{item['price']:>10.2f}{item['sales']:>8}")
    print(f"\nTotal Inventory Value: {total_val:.2f}")

def main():
    inv = load()
    while True:
        print("\n=== Inventory System ===")
        print("1. Add Item 2. Sell Item")
        print("3. View All 4. Low Stock 5. Exit")
        ch = input("Choice: ")
        if ch=="1": add_item(inv)
        elif ch=="2": sell_item(inv)
        elif ch=="3": view_inventory(inv)
        elif ch=="4": low_stock_alert(inv)
        elif ch=="5": break

if __name__ == "__main__":
    main()
```

CHAPTER 14

Python Interview Preparation — 175 Q&A

BEGINNER QUESTIONS (100)

Q1. What is Python?

A: Python is a high-level, interpreted, general-purpose programming language known for its simplicity and readability. Created by Guido van Rossum in 1991.

Q2. What are Python's key features?

A: Interpreted, dynamically typed, garbage-collected, supports OOP/functional/procedural paradigms, huge standard library, cross-platform.

Q3. What is PEP 8?

A: Python Enhancement Proposal 8 — the official style guide for Python code. It defines naming conventions, indentation (4 spaces), line length (79 chars), etc.

Q4. What is the difference between Python 2 and Python 3?

A: Python 2 reached end-of-life in 2020. Key differences: print is a function in 3, division returns float in 3, unicode is default in 3. Always use Python 3.

Q5. What is a variable?

A: A named reference to a value stored in memory. In Python: `x = 10`

Q6. What are Python's data types?

A: int, float, str, bool, list, tuple, set, dict, NoneType, complex, bytes.

Q7. What is dynamic typing?

A: Python determines variable types at runtime, not at declaration. `x = 10` (int), `x = 'hello'` (str) — same variable, different types.

Q8. What is the difference between int and float?

A: int stores whole numbers (10, -5); float stores decimals (3.14, -0.5).

Q9. How do you get user input?

A: `name = input('Enter your name: ')` — always returns a string.

Q10. What does print() do?

A: Outputs text or values to the console.

Q11. What is a comment in Python?

A: Text ignored by the interpreter. Single-line: `# comment`. Multi-line: triple quotes.

Q12. What is None in Python?

A: A special object representing 'no value' or 'nothing'. Similar to null in other languages.

Q13. What is type conversion?

A: Changing from one data type to another: `int('5')`, `str(42)`, `float('3.14')`.

Q14. What is the % operator?

A: Modulus — returns the remainder of division. `10 % 3 = 1`.

Q15. What is the // operator?

A: Floor division — divides and rounds down to the nearest integer. `7 // 2 = 3`.

Q16. What is the ** operator?

A: Exponentiation (power). `2 ** 10 = 1024`.

Q17. What is the difference between == and is?

A: `==` compares values; `is` compares identity (same object in memory). `[1,2] == [1,2]` is True; `[1,2] is [1,2]` is False.

Q18. What are comparison operators?

A: `==`, `!=`, `>`, `<`, `>=`, `<=` — all return True or False.

Q19. What are logical operators?

A: `and`, `or`, `not` — combine boolean expressions.

Q20. What is an if statement?

A: Conditional execution: code runs only if the condition is True.

Q21. What is elif?

A: Else-if: checks another condition if the previous one was False.

Q22. What is a for loop?

A: Iterates over a sequence (list, range, string, etc.) a fixed number of times.

Q23. What is a while loop?

A: Repeats as long as its condition remains True.

Q24. What does break do?

A: Exits the nearest enclosing loop immediately.

Q25. What does continue do?

A: Skips the rest of the current iteration and moves to the next.

Q26. What is range()?

A: Generates a sequence of integers. `range(start, stop, step)`. `stop` is excluded.

Q27. What is a function?

A: A named, reusable block of code defined with `def`.

Q28. What is the return statement?

A: Sends a value back to the caller. A function without `return` returns `None`.

Q29. What is a parameter vs argument?

A: Parameter: variable in function definition. Argument: actual value passed when calling.

Q30. What is a default parameter?

A: A parameter with a default value: `def f(x, y=10)`. If `y` is not passed, it is 10.

Q31. What is a list?

A: An ordered, mutable collection of items: `[1, 'hello', 3.14]`. Supports duplicates.

Q32. What is a tuple?

A: An ordered, immutable collection: `(1, 2, 3)`. Faster than list; can't be modified.

Q33. What is a set?

A: An unordered collection of unique items: `{1, 2, 3}`. No duplicates allowed.

Q34. What is a dictionary?

A: A collection of key-value pairs: `{'name': 'Alice', 'age': 30}`. Keys must be unique.

Q35. How do you add an item to a list?

A: `list.append(item)` — adds to end. `list.insert(index, item)` — inserts at position.

Q36. How do you remove an item from a list?

A: `list.remove(value)` — by value. `list.pop(index)` — by index.

Q37. What is list slicing?

A: Extracting a portion: `lst[start:stop:step]`. `lst[1:4]` gives indices 1, 2, 3.

Q38. How do you check if an item is in a list?

A: Using `in`: `'apple' in ['apple','banana']` → `True`.

Q39. How do you sort a list?

A: `list.sort()` — in-place. `sorted(list)` — returns new sorted list.

Q40. What is `len()`?

A: Returns the number of elements in a sequence.

Q41. How do you reverse a string?

A: `s[::-1]` — using slice with step -1.

Q42. What is string concatenation?

A: Joining strings with `+`: `'Hello' + ' ' + 'World'` → `'Hello World'`

Q43. What is an f-string?

A: Formatted string literal (Python 3.6+): `f'My name is {name}'`. Clean and fast.

Q44. What does `str.split()` do?

A: Splits a string into a list of substrings: `'a,b,c'.split(',')` → `['a','b','c']`.

Q45. What does `str.join()` do?

A: Joins a list into a string: `','.join(['a','b','c'])` → `'a,b,c'`.

Q46. What does `str.strip()` do?

A: Removes leading/trailing whitespace (or specified chars).

Q47. How do you convert a list to a set?

A: `set(my_list)` — automatically removes duplicates.

Q48. What is `dict.get()`?

A: Safely retrieves a value; returns `None` (or a default) if key doesn't exist — avoids `KeyError`.

Q49. How do you loop through a dictionary?

A: `for key, value in d.items()`. Or `for key in d, for value in d.values()`.

Q50. What is a nested list?

A: A list containing other lists: `[[1,2],[3,4],[5,6]]`. Access: `matrix[row][col]`.

Q51. What is scope?

A: The region where a variable is accessible. Local (inside function) vs Global (module level).

Q52. What is the global keyword?

A: Declares that a variable inside a function refers to the global scope variable.

Q53. What is a lambda function?

A: An anonymous one-line function: `lambda x: x*2`.

Q54. What is recursion?

A: A function that calls itself. Needs a base case to prevent infinite recursion.

Q55. What is a module?

A: A `.py` file containing Python code (functions, classes, variables) that can be imported.

Q56. How do you import a module?

A: `import math` or `from math import sqrt`.

Q57. What is pip?

A: Python's package installer. `pip install requests` installs the requests library.

Q58. What is a virtual environment?

A: An isolated Python environment per project, preventing package version conflicts.

Q59. What is exception handling?

A: Using try/except to gracefully handle runtime errors instead of crashing.

Q60. What does finally do?

A: Code in finally always runs, whether an exception occurred or not. Used for cleanup.

Q61. What is a class?

A: A blueprint for creating objects. Defined with the class keyword.

Q62. What is an object?

A: An instance of a class — a concrete entity with attributes and methods.

Q63. What is `__init__`?

A: The constructor method called automatically when an object is created.

Q64. What is `self`?

A: A reference to the current instance. First parameter of every instance method.

Q65. What is inheritance?

A: A child class inheriting attributes and methods from a parent class.

Q66. What is method overriding?

A: A child class providing its own implementation of a method defined in the parent.

Q67. What is encapsulation?

A: Hiding internal data using private attributes (`_` or `__`) and exposing controlled access.

Q68. What is polymorphism?

A: Different objects responding differently to the same method call.

Q69. What is a list comprehension?

A: `[expr for item in iterable if condition]` — concise way to create lists.

Q70. What does `enumerate()` do?

A: Returns (index, value) pairs from an iterable. `for i, v in enumerate(lst)`.

Q71. What does `zip()` do?

A: Pairs elements from multiple iterables: `zip([1,2],[3,4]) → (1,3),(2,4)`.

Q72. What does `map()` do?

A: Applies a function to every item: `list(map(str, [1,2,3])) → ['1','2','3']`.

Q73. What does `filter()` do?

A: Filters items where function returns True: `list(filter(lambda x: x>0, [-1,2,-3,4]))`.

Q74. What is the difference between `append` and `extend`?

A: `append` adds one item. `extend` adds all items from another iterable.

Q75. What is string immutability?

A: Strings cannot be changed in-place. Every modification creates a new string object.

Q76. How do you open and read a file?

A: with `open('file.txt', 'r')` as `f`: `content = f.read()`

Q77. How do you write to a file?

A: with `open('file.txt', 'w')` as `f`: `f.write('Hello')`

Q78. What is JSON?

A: JavaScript Object Notation — a text format for data exchange. Python handles it with the `json` module.

Q79. What is the difference between mutable and immutable?

A: Mutable (list, dict, set): can be changed in-place. Immutable (str, tuple, int): cannot be changed.

Q80. What is *args?

A: Allows a function to accept any number of positional arguments as a tuple.

Q81. What is **kwargs?

A: Allows a function to accept any number of keyword arguments as a dictionary.

Q82. What does pass do?

A: A null statement — placeholder when a block is required but no code is needed yet.

Q83. What is a docstring?

A: A string literal at the start of a function/class/module documenting its purpose.

Q84. How do you check the type of a variable?

A: type(x) returns the type. isinstance(x, int) checks if x is an int.

Q85. What is the in operator?

A: Tests membership: 'a' in 'apple' → True. 3 in [1,2,3] → True.

Q86. What is ternary operator in Python?

A: result = value_if_true if condition else value_if_false

Q87. What is a generator?

A: A function using yield that produces values one at a time, lazily.

Q88. What does next() do?

A: Retrieves the next value from an iterator/generator.

Q89. What is the difference between is and ==?

A: is checks object identity (same memory); == checks value equality.

Q90. What does id() return?

A: The unique memory address of an object.

Q91. What is the difference between del and remove?

A: del removes by index: del lst[0]. remove() removes by value: lst.remove(5).

Q92. How do you create an empty dictionary?

A: d = {} or d = dict()

Q93. What is negative indexing?

A: Access from the end: lst[-1] is the last item, lst[-2] is second to last.

Q94. What is the math module used for?

A: Mathematical functions: math.sqrt(), math.ceil(), math.floor(), math.pi, etc.

Q95. How do you generate a random number?

A: import random; random.randint(1, 100) for int; random.random() for float.

Q96. What is os.path.exists()?

A: Returns True if a file or directory path exists on the filesystem.

Q97. What is the difference between readline() and readlines()?

A: readline() reads one line. readlines() reads all lines into a list.

Q98. What is string repetition?

A: 'abc' * 3 → 'abcbcabcb'. Works for lists too: [0]*5 → [0,0,0,0,0].

Q99. How do you convert a string to uppercase?

A: s.upper(). To lowercase: s.lower(). To title case: s.title().

Q100. What is a syntax error vs a runtime error?

A: Syntax error: invalid code structure caught before running (missing colon, etc.). Runtime error: valid syntax that fails during execution (ZeroDivisionError, etc.).

INTERMEDIATE QUESTIONS (50)**Q101. What is a decorator?**

A: A function that wraps another function to add behaviour without changing its code. Applied with `@decorator_name`.

Q102. What is the difference between `@staticmethod` and `@classmethod`?

A: `staticmethod`: no access to class/instance; receives no implicit first arg. `classmethod`: receives `cls` (the class itself) as first arg.

Q103. What is a context manager?

A: An object that supports the `with` statement, managing setup/teardown via `__enter__` and `__exit__`.

Q104. What is list vs tuple performance difference?

A: Tuples are faster for creation and iteration; lists are faster for insertions. Tuples use less memory.

Q105. What is a generator expression?

A: `(expr for item in iterable)` — like list comprehension but lazy; uses $O(1)$ memory.

Q106. What is `deepcopy` vs `copy`?

A: `copy.copy()` creates a shallow copy (nested objects still shared). `copy.deepcopy()` copies everything recursively.

Q107. Explain the GIL (Global Interpreter Lock).

A: A mutex in CPython allowing only one thread to execute Python bytecode at a time. Limits true multi-threading for CPU-bound tasks; use multiprocessing instead.

Q108. What is `__slots__`?

A: Declares fixed attributes for a class, reducing memory usage by preventing `__dict__` creation per instance.

Q109. What is `*args` vs `**kwargs` unpacking?

A: At call site: `func(*[1,2,3])` unpacks list as positional args; `func(**{'a':1})` unpacks dict as keyword args.

Q110. What is a metaclass?

A: A class whose instances are classes. `type` is Python's default metaclass. Used for advanced class customisation.

Q111. What is memoisation?

A: Caching function results for given inputs to avoid redundant computation. Python provides `functools.lru_cache`.

Q112. What is the difference between `__str__` and `__repr__`?

A: `__str__`: human-readable string (for print). `__repr__`: unambiguous string for developers (for debugging). If only `__repr__` defined, it serves both.

Q113. What are abstract classes?

A: Classes that cannot be instantiated; define a common interface for subclasses. Use `abc.ABC` and `@abstractmethod`.

Q114. What is multiple inheritance?

A: A class inheriting from more than one parent: `class C(A, B)`. Python uses MRO (C3 linearisation) to resolve method order.

Q115. What is MRO?

A: Method Resolution Order — the order Python searches for methods in inheritance. View with `ClassName.__mro__`.

Q116. What is `__call__`?

A: Makes an object callable like a function: `obj()` invokes `obj.__call__()`.

Q117. What is a property?

A: The `@property` decorator makes a method accessible like an attribute, allowing getter/setter/deleter.

Q118. What is monkey patching?

A: Dynamically modifying a class or module at runtime. Powerful but can lead to confusing code.

Q119. What is the collections module?

A: Provides specialised containers: `Counter`, `defaultdict`, `OrderedDict`, `deque`, `namedtuple`.

Q120. What is `Counter`?

A: `from collections import Counter`. Counts hashable objects: `Counter('banana') → {'a':3,'n':2,'b':1}`.

Q121. What is `defaultdict`?

A: A dict that returns a default value for missing keys: `defaultdict(int)` returns 0 for missing keys.

Q122. What is `deque`?

A: Double-ended queue — $O(1)$ appends/pops from both ends. Faster than list for queues.

Q123. What is `namedtuple`?

A: A tuple subclass with named fields: `Point = namedtuple('Point', ['x', 'y'])`; `p=Point(1,2)`; `p.x → 1`.

Q124. What is `itertools.chain()`?

A: Chains multiple iterables together: `chain([1,2],[3,4]) → 1 2 3 4`.

Q125. What is `itertools.product()`?

A: Cartesian product of iterables — useful for nested loop replacement.

Q126. What is a regular expression?

A: A pattern for matching text. Python's `re` module: `re.findall(r'\d+', '12 cats and 3 dogs') → ['12','3']`.

Q127. What is `re.compile()`?

A: Compiles a regex pattern into a reusable object for better performance when used many times.

Q128. What is the difference between `re.match` and `re.search`?

A: `match()`: checks only at the beginning of the string. `search()`: scans through the entire string.

Q129. What is a closure?

A: A function that remembers the variables from its enclosing scope even after that scope has finished.

Q130. What is a partial function?

A: `functools.partial` — creates a new function with some arguments pre-filled.

Q131. What is `functools.reduce()`?

A: Applies a function cumulatively: `reduce(lambda a,b: a+b, [1,2,3,4]) → 10`.

Q132. What is the difference between `sorted()` and `sort()`?

A: `sorted()` returns a new list; works on any iterable. `sort()` modifies the list in-place; only for lists.

Q133. What is unpacking?

A: Assigning iterable values to variables: `a, b, c = [1, 2, 3]`. Extended: `first, *rest = [1,2,3,4]`.

Q134. What is a dataclass?

A: `from dataclasses import dataclass`. Auto-generates `__init__`, `__repr__`, `__eq__` based on field annotations.

Q135. What is type hinting?

A: Annotating function signatures and variables: `def add(a: int, b: int) -> int`. Checked by tools like `mypy`; not enforced at runtime.

Q136. What is the walrus operator (`:=`)?

A: Assignment expression (Python 3.8+): `while chunk := f.read(1024): process(chunk)`. Assigns and tests in one step.

Q137. What is match-case (structural pattern matching)?

A: Python 3.10+ feature — like switch/case but more powerful. match command: case 'quit': ...

Q138. What is the difference between threading and multiprocessing?

A: threading: shared memory, limited by GIL, good for I/O tasks. multiprocessing: separate processes, true parallelism, good for CPU tasks.

Q139. What is async/await?

A: Asynchronous programming model. async def creates a coroutine; await pauses it until the awaited task completes. Requires asyncio.

Q140. What is asyncio?

A: Python's standard library for async programming. Provides an event loop to run coroutines concurrently.

Q141. What is pickle?

A: Python's serialisation module — converts objects to bytes (pickling) and back (unpickling). Not safe for untrusted data.

Q142. What is the abc module?

A: Abstract Base Classes module. Use ABC and @abstractmethod to define interfaces.

Q143. What is __all__ in a module?

A: A list defining which names are exported when from module import * is used.

Q144. What is __name__ == '__main__'?

A: True only when the script is run directly (not imported). Used to guard test/demo code.

Q145. What is the difference between extend and update for dicts?

A: dict.update() merges another dict. {**d1, **d2} also merges (Python 3.5+). d1 | d2 merges in Python 3.9+.

Q146. What is the difference between is None and == None?

A: Use is None — it checks identity. == None also works but is not recommended per PEP 8.

Q147. What is enumerate()?

A: Returns (index, value) pairs: for i, v in enumerate(['a','b','c']): gives (0,'a'),(1,'b'),(2,'c').

Q148. What is zip_longest()?

A: itertools.zip_longest() — like zip but continues until the longest iterable is exhausted, filling with a fillvalue.

Q149. What is a sentinel value?

A: A special value used to signal end-of-data or a boundary condition (e.g., None, -1, float('inf')).

Q150. What is sys.argv?

A: A list of command-line arguments. sys.argv[0] is the script name; sys.argv[1:] are user-provided arguments.

ADVANCED QUESTIONS (25)**Q151. Explain Python's memory management.**

A: CPython uses reference counting + cyclic garbage collector. When refcount → 0, memory freed. Cyclic GC handles circular references using generational collection.

Q152. What is the difference between __new__ and __init__?

A: __new__ creates the object (allocates memory); __init__ initialises it. __new__ is called first. Override __new__ to control object creation (singleton pattern).

Q153. What is a descriptor?

A: An object that defines __get__, __set__, or __delete__. Used by property, classmethod, staticmethod internally.

Q154. Explain Python's data model.

A: Python's data model defines how objects interact with language features via dunder methods (__len__, __add__, __iter__, etc.).

Q155. What is the difference between slots and dict?

A: `__dict__` stores instance attributes dynamically (flexible, more memory). `__slots__` uses a fixed array (less memory, faster attribute access).

Q156. What is a coroutine vs a thread?

A: Coroutines are cooperative (yield control explicitly via `await`); threads are preemptive (OS switches them). Coroutines are lighter and avoid GIL issues.

Q157. What is the event loop in asyncio?

A: A single-threaded scheduler that runs coroutines. It polls for I/O events and resumes the correct coroutine when data is ready.

Q158. What is a context variable (contextvars)?

A: Python 3.7+. Like thread-local storage but for coroutines — allows per-task context without explicit passing.

Q159. What is monkey-patching and when is it dangerous?

A: Replacing methods/attributes at runtime. Dangerous because it breaks expected interfaces, causes hard-to-debug issues, and couples code to internals.

Q160. Explain the MRO for diamond inheritance.

A: Python uses C3 linearisation: class `D(B,C)`, `B(A)`, `C(A)` → MRO: `D,B,C,A`. Guarantees each class appears once and respects local precedence.

Q161. What is `__init_subclass__`?

A: A hook called when a class is subclassed. Useful for registering subclasses or enforcing constraints without metaclasses.

Q162. What is a weakref?

A: A reference to an object that doesn't prevent garbage collection. Used to avoid memory leaks in caches: `import weakref; ref = weakref.ref(obj)`.

Q163. What is the buffer protocol?

A: An interface allowing objects to expose their internal data as a memory buffer — used by NumPy arrays, bytes, `memoryview` for zero-copy data access.

Q164. What is CPython vs PyPy?

A: CPython is the reference implementation in C. PyPy uses JIT compilation and is often 5-10x faster for CPU-bound code. Trade-offs: compatibility, startup time.

Q165. What is the import system internals?

A: Python searches `sys.path`, checks `sys.modules` cache, runs the module file, caches it. Custom importers can be added via `importlib`.

Q166. What is `__future__`?

A: Allows use of features from future Python versions in current code: `from __future__ import annotations`.

Q167. What is the difference between `exec` and `eval`?

A: `eval()` evaluates a single expression and returns the result. `exec()` executes statements. Both can be security risks with untrusted input.

Q168. What is a frozen set?

A: An immutable set: `frozenset({1,2,3})`. Can be used as dictionary keys or set elements.

Q169. What is `__getattr__` vs `__getattribute__`?

A: `__getattr__` is called only when the attribute is not found normally. `__getattribute__` is called on every attribute access — override with care.

Q170. Explain Python's integer caching.

A: CPython caches small integers (-5 to 256) and short strings — they're singletons. `a = 256; b = 256; a is b` → `True`. `a = 257; b = 257; a is b` → `False` (implementation detail).

Q171. What is a thread-safe queue?

A: `queue.Queue` — FIFO thread-safe queue. Internally uses a lock. Used for producer-consumer patterns across threads.

Q172. What is multiprocessing.Pool?

A: A pool of worker processes. `pool.map(func, iterable)` distributes work across processes for true parallelism.

Q173. What is Cython?

A: A superset of Python that compiles to C. Provides C-like speed while maintaining Python syntax. Used for performance-critical extensions.

Q174. What is the difference between yield and yield from?

A: `yield` produces a value. `yield from` delegates to a sub-generator, forwarding values bidirectionally — simplifies generator composition.

Q175. What is structural pattern matching (match-case)?

A: Python 3.10+ feature for matching objects by structure: `match point: case Point(x=0, y=0): print('origin')`. Supports guards, OR patterns, capture patterns.

CHAPTER 15

Quick Revision Guide & Cheat Sheets

Python Syntax Cheat Sheet

```
# Variables & Types
x = 10; y = 3.14; s = "hi"; b = True; n = None

# Input/Output
name = input("Name: "); print(f"Hello {name}")

# Operators
+ - * / // % ** # arithmetic
== != > < >= <= # comparison
and or not # logical

# Conditions
if x > 0:
...
elif x == 0:
...
else:
...

# Loops
for i in range(10): ...
while condition: ...
break / continue / pass

# Functions
def func(a, b=10, *args, **kwargs):
    return a + b
f = lambda x: x**2

# List / Dict / Set Comprehensions
[x**2 for x in range(10) if x%2==0]
{k: v for k,v in d.items()}
{x for x in range(5)}
```

Data Structures Cheat Sheet

```
# List
lst = [1, 2, 3]
lst.append(4); lst.insert(0,0); lst.remove(2)
lst.sort(); lst.reverse(); lst.pop()
len(lst); lst[1:3]; 2 in lst

# Tuple
t = (1, 2, 3); a, b, c = t
t.count(1); t.index(2)
```

```
# Set
s = {1,2,3}; s.add(4); s.remove(3)
s1 | s2; s1 & s2; s1 - s2

# Dictionary
d = {'a': 1, 'b': 2}
d['c'] = 3; d.get('x', 0)
del d['a']
d.keys(); d.values(); d.items()
for k, v in d.items(): ...

# String
s = "hello world"
s.upper(); s.lower(); s.strip(); s.split()
s.replace('l','r'); s.find('o'); s.count('l')
f"{name} is {age}"
```

OOP Cheat Sheet

```
class Animal:
    species = "Mammal" # class attr
    def __init__(self, name): # constructor
        self.name = name # instance attr
    def speak(self): # method
        return f"{self.name} speaks"
    @classmethod
    def create(cls, name): return cls(name)
    @staticmethod
    def breathes(): return True
    @property
    def info(self): return self.name
    def __str__(self): return self.name
    def __repr__(self): return f"Animal({self.name})"

class Dog(Animal): # inheritance
    def speak(self): # override
        return f"{self.name}: Woof"
    def __init__(self, name, breed):
        super().__init__(name) # call parent
        self.breed = breed
```

File & Error Handling Cheat Sheet

```
# Files
with open('f.txt', 'r') as f: data = f.read()
with open('f.txt', 'w') as f: f.write('hello')
with open('f.txt', 'a') as f: f.write('more')

# Exceptions
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(e)
```

```

except (TypeError, ValueError):
    print("Type or Value error")
else:
    print("No error")
finally:
    print("Always runs")

raise ValueError("custom message")

class MyError(Exception): pass

```

Common Built-in Functions

Function	What It Does
print(*args)	Output to console
input(prompt)	Get user input (returns str)
len(obj)	Length of sequence
range(s,e,step)	Integer sequence
type(obj)	Return type of object
int/float/str/bool	Type conversion
list/tuple/set/dict	Collection constructors
sorted(iter, key)	Return new sorted list
enumerate(iter)	Return (index, value) pairs
zip(*iters)	Pair elements from iterables
map(func, iter)	Apply function to each item
filter(func, iter)	Keep items where func is True
sum/min/max(iter)	Aggregate functions
abs(n)	Absolute value
round(n, digits)	Round to n digits
open(path, mode)	Open a file
dir(obj)	List attributes/methods
help(obj)	Show documentation
id(obj)	Memory address
isinstance(obj,cls)	Check type membership

One-Page Chapter Summaries

Ch.1 Introduction

- Programming = instructions for computers
- Python: readable, versatile, in-demand
- Think algorithmically: Input → Process → Output

Ch.2 Fundamentals

- Variables store values; types: int, float, str, bool
- input() always returns str — convert if needed
- f-strings: f'Hello {name}' — cleanest formatting

Ch.3 Decision Making

- if/elif/else control program flow
- == compares values; = assigns
- and/or/not combine conditions

Ch.4 Loops

- for: fixed iterations; while: condition-based
- break exits loop; continue skips iteration
- Nested loops: $O(n^2)$ — use carefully

Ch.5 Functions

- def name(params): return value
- *args = variable positional; **kwargs = variable keyword
- Lambda: anonymous one-liner function

Ch.6 Data Structures

- List: ordered, mutable, duplicates OK
- Dict: key-value pairs, fast lookup
- Set: unique items, fast membership test
- Tuple: immutable, use for fixed data

Ch.7 Strings

- Immutable sequences — every operation creates new string
- Rich method library: split, join, strip, replace
- f-strings are fastest for formatting

Ch.8 OOP

- Class = blueprint; Object = instance
- `__init__` is the constructor
- Inheritance enables code reuse
- `self` refers to the current instance

Ch.9 Files & Errors

- Always use `'with open()'` — auto-closes
- `try/except/else/finally` handles runtime errors
- Create custom exceptions for domain logic

Ch.10 Modules

- `import module` / `from module import name`
- `pip` installs third-party packages
- Virtual environments isolate project dependencies

Ch.11 Data Analysis

- NumPy: fast array operations (vectorised)
- Pandas: DataFrame for tabular data
- `pd.read_csv()` and `df.describe()` are your starting points

Ch.12 Advanced

- List comprehensions: concise and fast
- Generators: lazy, memory-efficient — use `yield`
- Decorators: wrap functions to add cross-cutting behaviour

Congratulations — You Have Completed the Python Complete Handbook!

You now have the knowledge to write Python programs, solve coding problems, build real-world projects, analyse data, and confidently walk into a Python interview. The next step is practice — build something every day.

Keep coding. Keep building. Keep growing.